

astrid-runtime/astrid

Astrid : un runtime de sécurité Rust très outillé en CI, mais 42 bugs ouverts, un seul mainteneur et des tests natifs difficiles à rejouer localement

Astrid is a portable, capability-secure operating system for composable software.

39

FICHIERS DE TEST
cargo test, insta

1

WORKFLOWS CI

42

BUGS ÉTIQUETÉS

192

ISSUES OUVERTES

<https://github.com/astrid-runtime/astrid> Licence Apache-2.0 Onboard · profil qa

☑ Que fait ce projet et quels sont ses parcours critiques ?

VERDICT

Les mécanismes de sécurité sont indépendants et « fail closed » : chaque parcours CLI se teste d'abord en négatif (accès refusé attendu), sur macOS et Linux seulement, Windows étant hors des archives de la v2026.9.0 ^{1 2}.

Astrid est « a portable, capability-secure operating system for composable software » ³ : un runtime user-space sur macOS et Linux qui exécute des composants WebAssembly (« capsules ») dans un sandbox Wasmtime, sans autorité ambiante, chaque accès fichier/réseau/processus étant contrôlé par un jeton de capacité signé ¹. Projet jeune (créé le 2026-02-15 ⁴), 10277 étoiles ⁵, 216 issues ouvertes ⁶, dernier push le 2026-09-09 ⁷, version courante v2026.9.0 ².

Parcours critiques à couvrir, tirés du README ¹ :

- 1 Cycle de vie du daemon : `astrid start` / `status` / `capsule list` / `stop`, avec retrait de l'état dans `astrid.volume` à l'arrêt et restauration au redémarrage.
- 2 Initialisation d'une distribution : `astrid init --distro <source>` (repo, manifeste local, bundle signé `--offline`), écriture d'un `Distro.lock` épinglant chaque capsule par hash BLAKE3.
- 3 Cycle de vie des capsules à chaud : `capsule new` / `build` / `install` / `update` / `remove` sans redémarrage.
- 4 Isolation par principal : `agent create`, `caps show`, `quota set`, `pair-device issue --scope use-only` ; un principal ne doit jamais lire l'espace d'un autre, et le système « fails closed ».
- 5 Montage du stockage durable : `storage mount` / `status` / `sync` / `unmount` via FSKit (macOS 26+) ou FUSE (Linux).

Chaque parcours traverse les mécanismes de sécurité indépendants (sandbox WASM, manifeste allow-list, ACL IPC, jeton de capacité, gate d'approbation, sandbox OS, chaîne d'audit) : ce sont eux qu'il faut tester en négatif ¹. Windows est testé en CI mais absent des archives de la release ¹.

☑ Comment est-il testé aujourd'hui ?

VERDICT

39 fichiers de test relevés entre `crates/astrid-integration-tests`, `crates/astrid-test` **et** `e2e/` (`cargo test + snapshots insta`), **mais aucun taux de couverture dans le cache** ^{8 9 10}.

☐	<code>.cargo/</code>	config Cargo locale
☐	<code>.gemini/</code>	config agent Gemini
☐	<code>.github/</code>	20 workflows GitHub Actions
☐	<code>changes/</code>	notes de changement en attente
☐	<code>container/</code>	définitions de conteneur
☐	<code>crates/</code>	code principal : 33 crates Rust, 32 membres du workspace
☐	<code>docs/</code>	documentation
☐	<code>e2e/</code>	scénarios TOML et fixtures bout en bout
☐	<code>fuzz/</code>	tests de fuzzing
☐	<code>native/</code>	code natif macOS
☐	<code>release/</code>	config de release (nightly.toml)
☐	<code>scripts/</code>	scripts build, CI, release
☐	<code>.dockerignore</code>	exclusions du build Docker
☐	<code>.gitattributes</code>	attributs git
☐	<code>.gitignore</code>	exclusions git
☐	<code>.gitmodules</code>	sous-modules git
☐	... et 12 autres entrées, voir <code>sources/tree.md</code>	

Pas de dossier `test/`, `tests/` ou `spec/` à la racine : les tests vivent sous `crates/` et `e2e/` ¹⁰. Le crate dédié est `crates/astrid-integration-tests` ¹¹, 39 fichiers relevés ⁸, avec `cargo test` et des snapshots `insta` ⁹; un crate `astrid-test` fournit `harness.rs` et `mocks.rs` ^{10 12}.

Bout en bout : `e2e/` contient des scénarios TOML (`capability-scenarios.toml`, `cli-scenarios.toml`, `http-scenarios.toml`, `runtime-scenario-specs.toml`, `waiter-surfaces.toml`, `first-party-capsule-scenarios.toml`), un `concurrency.sh` et des fixtures de capsules adverses et concurrentes ^{13 10}. Un dossier `fuzz/` existe à la racine ¹⁴.

Commande locale : `cargo test --workspace -- --quiet` ¹⁵; en CI, `cargo test --workspace --locked` sur ubuntu et `scripts/ci/test-workspace-macos.sh` sur macos ^{10 16}.

Règle d'équipe : « Tests required. New features need tests. Bug fixes need a regression test » ¹⁷.

Le cache ne contient ni taux de couverture, ni nombre de cas dans les scénarios TOML, ni le contenu de `runtime-e2e.yml` (« non lu ») ; à vérifier dans <https://github.com/astrid-runtime/astrid/tree/main/e2e> et <https://github.com/astrid-runtime/astrid/blob/main/github/workflows/runtime-e2e.yml> ¹⁰.

☑ Que dit la CI et quand tourne-t-elle ?

VERDICT

Un seul workflow lu, `CI`, sur `push` et `pull_request`, 14 jobs dont les tests sur `ubuntu` et `macos` ; les 19 autres workflows et l'état des runs restent à vérifier sur GitHub ^{18 19 20 16 21}.

Un workflow principal, `CI` (`.github/workflows/ci.yml`), déclenché sur `push` et `pull_request` ^{18 22 19 20}. Le push ne déclenche que sur `main` et `stream-*` et seulement si des `.rs`, `Cargo.toml` / `Cargo.lock`, des scripts de release ou certains workflows changent ; toute PR déclenche sans filtre (« Required build/test checks must also run for stacked and docs-only PRs ») ¹⁶.

14 jobs, toolchain Rust 1.95 épinglée, `harden-runner` sur chaque job ¹⁶ :

- ❑ Qualité : `fmt`, `clippy -D warnings`, `check --all-features`, `msrv` (1.95), `audit` (cargo-audit 0.22.2).
- ❑ Tests : `test` (`ubuntu` + `macos`, `CARGO_PROFILE_TEST_DEBUG=0`, cf. #954), `linux-fuse-e2e` (un test `--ignored` de montage FUSE natif), `linux-v0104-upgrade` (script de migration depuis v0.10.4), `windows-filesystem-native` (WinFsp 2.1, x86_64 et aarch64), `wasm-portability`.
- ❑ Release : `linux-release-smoke` (4 cibles gnu/musl, contrôles glibc ≤ 2.34 et ELF statique).
- ❑ PR seulement, consultatifs : `api-compatibility` (cargo-semver-checks, cf. #1480), `public-rust-api-diff`, `gateway-openapi-contract`.

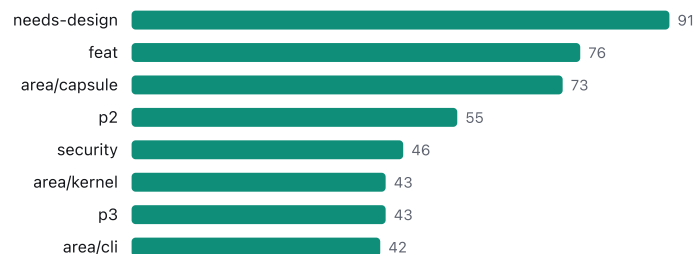
Au total 20 workflows dans `.github/` ²¹, dont `runtime-e2e.yml`, `native-storage-certification.yml`, `supervised-storage-certification.yml`, `storage-benchmark.yml`, `windows-local-transport.yml`, `codeql.yml`, `nightly.yml` ¹⁶. Le risque CI est évalué « low » ^{23 24}.

Le cache ne contient ni l'état des derniers runs ni les déclencheurs des 19 autres workflows (le MCP n'expose pas les runs Actions) ; à vérifier dans <https://github.com/astrid-runtime/astrid/actions>.

☑ Où sont les bugs connus ?

VERDICT

42 issues bug sur 192 ouvertes, 36 en p1, concentrées sur area/capsule (73) et area/kernel (43) ; **91 issues** needs-design n'ont pas encore de comportement à tester ^{25 26 27 28 29 30}.



192 issues ouvertes ²⁶, 149 fermées sur 30 jours ³¹. Étiquettes utiles pour un plan de test : bug ^{42 25}, security ^{46 32}, test ^{16 33}, blocked ^{16 34} ; priorités p1 ^{36 27}, p2 ^{55 35}, p3 ^{43 36}.

Zones les plus chargées : area/capsule ^{73 28}, area/kernel ^{43 29}, area/cli ^{42 37}, area/runtime ^{25 38}, area/core ^{25 39}. 91 issues sont needs-design ³⁰ : comportement non figé, donc cas de test à écrire avec prudence.

Bugs concrets et reproductibles parmi les « good first » : #459 « Markdown parser in TUI fails on numbered lists >= 10 » ^{40 41}, #276 « MCP interceptor: propagate tool_result.is_error instead of treating it as success » ^{42 43}, #461 « is_known_tag coverage table is 18 vs 19 and omits ToolCancelRequest » ^{44 45}, #476 sur l'enregistrement dupliqué dans CapsuleRegistry ⁴⁶.

L'issue la plus discutée est un ticket de test : #1708 « test(container): qualify the signed amd64 Linux-backed Astrid host profile », 29 commentaires ^{47 48 49}.

Le cache ne contient pas le croisement bug × area/* ni la liste des 42 bugs ; à vérifier dans <https://github.com/astrid-runtime/astrid/issues?q=is%3Aopen+label%3Abug>.

☑ Comment reproduire et signaler un bug ici ?

VERDICT

Pas d'issue sans reproduction (version, étapes, attendu et observé), pas de PR sans issue assignée ni test de régression, jamais de vulnérabilité en issue publique ; 24 PR ouvertes attendent déjà ^{17 50}.

Procédure imposée par CONTRIBUTING.md ¹⁷ :

- 1 Chercher une issue existante, puis utiliser le template « bug report ».
- 2 Reproduire avant d'ouvrir : version ou commit affecté, étapes exactes, comportement attendu et observé, preuves. Les audits en masse, findings spéculatifs et rapports générés par IA non vérifiés sont refusés (« AI output is a drafting aid, not evidence »).
- 3 Vulnérabilité : jamais d'issue publique, passer par GitHub Security Advisories ¹⁷ ; une politique SECURITY.md existe ⁵¹.
- 4 Pour corriger soi-même : issue assignée d'abord (« Every PR must be linked to an issue. No exceptions »), régression obligatoire pour un bug fix, commit `git commit -s` (DCO), fragment `changes/{issue}.fixed.md`, titre Conventional Commits, template PR complet sinon rejet CI ¹⁷.

Le flux est actif : 24 PR ouvertes ⁵⁰ et une longue liste de correctifs fusionnés en 30 jours, presque tous en `fix(...)`, par exemple #1906 « fix(init): honor explicit grants after completed batch resume » ^{52 53} et #1870 « fix(test): provision private Windows run directory » ⁵⁴. Une PR de test récente donne le modèle d'une fixture négative : #1849 « test(capsule): use an unsatisfiable astrid-version fixture » ^{55 56}.

Le cache ne contient pas le contenu des templates d'issue (`.github/ISSUE_TEMPLATE`) ; à vérifier dans <https://github.com/astrid-runtime/astrid/tree/main/.github>.

☑ Quelles zones sont peu couvertes ?

VERDICT

Sans mesure de couverture, les zones à risque se déduisent : bus factor 1 (309 commits pour `joshuajbouw`), Windows hors release, montage FUSE réduit à un test `--ignored`, 16 issues `test` et 16 `blocked` ^{57 58 1 16 33 34}.

Le cache ne contient aucune mesure de couverture de code ; à vérifier dans <https://github.com/astrid-runtime/astrid/actions> (job `test`) ¹⁰. Les signaux ci-dessous sont indirects.

- ❑ Concentration humaine : bus factor 1 ⁵⁷, `joshuajbouw` 309 commits contre 7 pour le second contributeur ^{58 59} ; risque « high » ^{60 61}. Le relecteur et l'auteur des tests sont la même personne : la revue QA externe a de la valeur.
- ❑ Rythme soutenu sans creux : 41 commits en W26 ⁶², 17 en W37 ⁶³, dernier commit le 2026-09-09 ⁶⁴ : les régressions arrivent vite, rejouer les scénarios `e2e/` à chaque release.
- ❑ Plateformes : Windows testé en CI mais hors release ¹ ; le montage FUSE natif est un seul test `--ignored` en CI ¹⁶ ; FSKit exige macOS 26+ et une approbation d'extension signée ¹, difficile à automatiser.
- ❑ Dépendances : 117 entrées dans `Cargo.toml`, 90 externes, 8 épinglées en exact ^{65 66} ; les deux PR Dependabot ouvertes (#1834, #1877) ^{67 68} sont des candidats à une passe de non-régression.
- ❑ Fonctionnel : 16 issues `test` ³³ et 16 `blocked` ³⁴ ; les thèmes chauds `campaign/os-universal` (32 issues ⁶⁹) et les décisions en attente (#1664 « specify the first-owner enrollment ceremony » ⁷⁰, #1692 ⁷¹) n'ont pas encore de comportement spécifié à tester.
- ❑ Le contenu de `fuzz/` et des 19 workflows hors `ci.yml` n'a pas été lu ^{10 16}.



Plan de test en 5 points

- 1 Fumée sur macOS et Linux : `cargo test --workspace -- --quiet` , puis le cycle `astrid start / status / capsule list / stop` en vérifiant que l'état est retiré dans `astrid.volume` à l'arrêt et restauré au redémarrage ^{15 1}.
- 2 Sécurité en négatif, risque haut : pour `astrid init --distro` , `capsule install` , `agent create` et `storage mount` , un cas sans jeton de capacité ou avec manifeste vide doit être refusé (« fails closed ») ; rejouer `e2e/capability-scenarios.toml` et les fixtures `e2e/fixtures/astrid-capsule-adversarial` ^{1 10}.
- 3 Régression sur les 42 `bug` ouverts, en commençant par les reproductibles : #459 (listes numérotées ≥ 10 dans le TUI), #276 (`is_error` MCP traité comme un succès), #461 (table `is_known_tag` 18 vs 19) ^{25 41 43 45}.
- 4 Plateformes, risque moyen : un passage complet sur macOS 26+ avec FSKit (approbation d'extension signée) et un sur Linux FUSE, où la CI ne joue qu'un test `--ignored` (`linux-fuse-e2e`) ; Windows est testé en CI mais hors archives, à traiter comme non supporté ^{1 16}.
- 5 Non-régression des dépendances et des mises à niveau : rejouer la suite sur les PR Dependabot #1834 et #1877, sur la migration depuis v0.10.4 (`linux-v0104-upgrade`), et suivre #1708 (29 commentaires) qui qualifie le profil hôte Linux ^{72 73 16 49 48}.

NOTES

✓ Sources

1 source du repo `readme.md`
2 donnée collectée `releases.0.tag`
3 donnée collectée `repo.description`
4 donnée collectée `repo.created_at`
5 donnée collectée `repo.stars`
6 donnée collectée `repo.open_issues`
7 donnée collectée `repo.pushed_at`
8 donnée collectée `tests.files`
9 donnée collectée `tests.framework`
10 source du repo `tests.md`
11 donnée collectée `tests.dir`
12 source du repo `tree.md`
13 donnée collectée `tree.20.role`
14 donnée collectée `tree.21.role`
15 donnée collectée `build.test`
16 source du repo `ci.md`
17 source du repo `contributing.md`
18 donnée collectée `build.ci.0.name`
19 donnée collectée `build.ci.0.triggers.0`
20 donnée collectée `build.ci.0.triggers.1`
21 donnée collectée `tree.4.role`
22 donnée collectée `build.ci.0.path`
23 donnée collectée `risks.3.level`
24 donnée collectée `risks.3.note`
25 donnée collectée `issues.by_label.bug`
26 donnée collectée `issues.open`
27 donnée collectée `issues.by_label.p1`
28 donnée collectée `issues.by_label.area/capsule`
29 donnée collectée `issues.by_label.area/kernel`

30 donnée collectée `issues.by_label.needs-design`
31 donnée collectée `issues.closed_30d`
32 donnée collectée `issues.by_label.security`
33 donnée collectée `issues.by_label.test`
34 donnée collectée `issues.by_label.blocked`
35 donnée collectée `issues.by_label.p2`
36 donnée collectée `issues.by_label.p3`
37 donnée collectée `issues.by_label.area/cli`
38 donnée collectée `issues.by_label.area/runtime`
39 donnée collectée `issues.by_label.area/core`
40 donnée collectée `issues.good_first.3.title`
41 donnée collectée `issues.good_first.3.url`
42 donnée collectée `issues.good_first.4.title`
43 donnée collectée `issues.good_first.4.url`
44 donnée collectée `issues.good_first.2.title`
45 donnée collectée `issues.good_first.2.url`
46 donnée collectée `issues.good_first.1.title`
47 donnée collectée `issues.hot.0.title`
48 donnée collectée `issues.hot.0.comments`
49 donnée collectée `issues.hot.0.url`
50 donnée collectée `pulls.open`
51 donnée collectée `business.security_policy`
52 donnée collectée `pulls.merged_30d.0.title`
53 donnée collectée `pulls.merged_30d.0.url`
54 donnée collectée `pulls.merged_30d.17.title`
55 donnée collectée `pulls.merged_30d.28.title`
56 donnée collectée `pulls.merged_30d.28.url`
57 donnée collectée `activity.bus_factor`
58 donnée collectée `activity.contributors.0.commits`

59	donnée collectée	<code>activity.contributors.1.commits</code>
60	donnée collectée	<code>risks.1.level</code>
61	donnée collectée	<code>risks.1.note</code>
62	donnée collectée	<code>activity.commits_per_week.0.count</code>
63	donnée collectée	<code>activity.commits_per_week.11.count</code>
64	donnée collectée	<code>activity.last_commit</code>
65	donnée collectée	<code>deps.count</code>
66	donnée collectée	<code>risks.2.note</code>
67	donnée collectée	<code>roadmap.open_prs.0.title</code>
68	donnée collectée	<code>roadmap.open_prs.1.title</code>
69	donnée collectée	<code>issues.by_label.campaign/os-universal</code>
70	donnée collectée	<code>issues.hot.6.title</code>
71	donnée collectée	<code>issues.hot.8.title</code>
72	donnée collectée	<code>roadmap.open_prs.0.url</code>
73	donnée collectée	<code>roadmap.open_prs.1.url</code>